Contents

update_F.R — Companion Document

Module: `code/update_F.R` **Project:** Supervised Bayesian Matrix Factorization (multiomicsGEP) **Role:** Variational update for the genomics loading matrix F (factor columns f_{.k})

Table of Contents

Overview

`update_F.R` implements the CAVI variational update for the factor loading matrix **F** (shape p × K) in the joint genomics + survival model. F appears **only** in the genomics likelihood and has no role in the Cox proportional hazards survival term — this structural fact drives the module's key mathematical simplification.

The update is a **vector EBNM** (Empirical Bayes Normal Means) problem solved independently for each factor column f_{.k}. Within each column, the p feature-wise posterior means and variances are computed in closed form, then passed to an EBNM solver that applies the chosen prior (default: point-normal) and returns posterior moments.

Depends on: `compute_R_k()` from `code/update_L.R` — source that file before this one.

Mathematical Background

Model Structure

The genomics observations follow:

```
Y ≈ L F'    (n × p matrix, L is n × K, F is p × K)
```

with feature-wise precision `Tau[j]` (heteroscedastic noise). The residual for factor k at feature j is:

```
R_k[i,j] = Y[i,j] - sum_{k' != k} EL[i,k'] * EF[j,k']
```

This is computed by `compute_R_k()` and excludes the current factor k (Gauss-Seidel schedule).

EBNM Sufficient Statistics for f_

For each feature j, the ELBO contribution from factor k leads to the following normal sufficient statistics:

Quantity	Formula	Shape	Notes
`A_F[j]`	`Tau[j] * sum(EL2_k)`	p-vector	`sum(EL2_k)` is a **scalar** broadcast across all j
`B_F[j]`	`Tau[j] * (t(R_k) %*% EL_k)[j]`	p-vector	Inner product of residual column with L posterior
`x_F[j]`	`B_F[j] / A_F[j]`	p-vector	EBNM observation (“sufficient statistic”)
`s_F[j]`	`1 / sqrt(A_F[j])`	p-vector	EBNM noise standard deviation

`sum(EL2_k)` is the sum of the second moments of the k-th column of L across all n samples:

```
sum_EL2_k = sum_i EL2[i,k]   (scalar, includes Var_q(l_{ik}) + EL[i,k]^2)
```

The Tau Cancellation Property

This is the **central mathematical insight** of the module.

Tau Cancels in x_j (observation)

```
x_j = B_F[j] / A_F[j]
    = (Tau[j] * (t(R_k) %*% EL_k)[j]) / (Tau[j] * sum_EL2_k)
    = (t(R_k) %*% EL_k)[j] / sum_EL2_k
```

The `Tau[j]` factors in numerator and denominator cancel exactly. The EBNM observation `x_j` is **identical regardless of feature precision**.

Tau Does NOT Cancel in s_j (noise level)

```
s_j = 1 / sqrt(A_F[j]) = 1 / sqrt(Tau[j] * sum_EL2_k)
```

Here `Tau[j]` remains under the square root. A feature with higher precision (larger `Tau[j]`) gets a **smaller** noise level `s_j`, which means the prior applies less shrinkage and the posterior mean stays closer to `x_j`.

Consequence

Features with the same residual signal `x_j` are differentially shrunk toward zero depending on their noise precision. Clean features (high Tau) are shrunk less; noisy features (low Tau) are shrunk more. This is the heteroscedastic shrinkage mechanism for F.

Tau	x_j	s_j	Shrinkage
1	same	large	strong
1000	same	small (~1/sqrt(1000) smaller)	weak

Demo 1 illustrates this with `max|x_j diff| ≈ 0` and `s_j ratio ≈ sqrt(1000)`.

API Reference

`update_F_k(Tau, EL_k, EL2_k, R_k, prior_family, A_floor)`

Computes the variational posterior for a single factor column f_{.k}.

Arguments:

Parameter	Type	Description
`Tau`	numeric vector, length p	Feature-wise noise precisions
`EL_k`	numeric vector, length n	Posterior means of L column k: E_q[l_{.k}]
`EL2_k`	numeric vector, length n	Posterior second moments of L column k: E_q[l_{.k}^2]
`R_k`	numeric matrix, n × p	Residual matrix excluding factor k (from `compute_R_k`)
`prior_family`	character	EBNM prior family; default `"point_normal"`
`A_floor`	numeric	Minimum value for A_F to avoid division by zero; default `1e-10`

Returns: Named list

Field	Type	Description
`mean`	numeric vector, length p	Posterior mean E_q[f_{.k}]
`second`	numeric vector, length p	Posterior second moment E_q[f_{.k}^2]
`sd`	numeric vector, length p	Posterior standard deviation
`A`	numeric vector, length p	EBNM precision A_F (after floor)
`B`	numeric vector, length p	EBNM numerator B_F

Field	Type	Description
x	numeric vector, length p	EBNM sufficient statistic x_F = B/A
s	numeric vector, length p	EBNM noise s_F = 1/sqrt(A)
`sum_EL2_k`	numeric scalar	sum(EL2_k), shared across all features
`ebnm_result`	list	Raw output from `ebnm()` call

Key implementation details:

- `A_F = pmax(Tau * sum(EL2_k), A_floor)` — the floor prevents NaN when Tau is zero
- `B_F = Tau * (t(R_k) %*% EL_k)` — matrix-vector product gives p-vector
- Second moment satisfies: `second[j] = sd[j]^2 + mean[j]^2`

`update_F_all(Y, EL, EL2, EF, EF2, Tau, prior_family, A_floor)`

Gauss-Seidel loop over all K factor columns.

Arguments:

Parameter	Type	Description
Y	numeric matrix, n × p	Observed genomics data
EL	numeric matrix, n × K	Posterior means of L (fully updated from L step)
EL2	numeric matrix, n × K	Posterior second moments of L
EF	numeric matrix, p × K	Current posterior means of F (updated in place)
EF2	numeric matrix, p × K	Current posterior second moments of F
Tau	numeric vector, length p	Feature-wise precisions
`prior_family`	character	EBNM prior family; default `"point_normal"`
`A_floor`	numeric	Floor for A; default `1e−10`

Returns: Named list

Field	Type	Description
EF	numeric matrix, p × K	Updated posterior means of F
EF2	numeric matrix, p × K	Updated posterior second moments of F
`details`	list of length K	Per-factor output from `update_F_k`

Important Gauss-Seidel note:

`update_F_all` passes the **fully updated** `EL` (not a stale copy) because the `L` step completes before the F step in the CAVI schedule. Within the F loop, `EF_curr` is updated column-by-column so that each `compute_R_k` call sees the most recent estimates of all other F columns. This is the Gauss-Seidel (sequential) schedule, as opposed to a Jacobi (parallel) schedule.

```
for k in 1:K:
    R_k  <- compute_R_k(Y, EL, EF_curr, k)   # excludes column k
    res  <- update_F_k(Tau, EL[,k], EL2[,k], R_k, ...)
    EF_curr[,k]  <- res$mean
    EF2_curr[,k] <- res$second
```

Test Suite

File: `tests/test_update_F.R` Total: **26 tests across 9 groups** — all passing.

T1: Mathematical Identity Checks (6 tests)

Test	What is verified
T1.1	`A_F[j] = Tau[j] * sum(EL2_k)` exactly
T1.2	`B_F[j] = Tau[j] * (t(R_k) %*% EL_k)[j]` exactly
T1.3	x is a p-vector (length p)
T1.4	Tau cancels in x_j: `Tau=1` vs `Tau=1000` produce identical `x_j`
T1.5	Tau does NOT cancel in s_j: `s_j` ratio is significantly different
T1.6	Second moment identity: `second[j] = sd[j]^2 + mean[j]^2`

T2: WLS / Non-Informative Limit (2 tests)

Test	What is verified
T2.1	When `EL2_k = EL_k^2` (zero posterior variance in L), `x_j` equals the OLS estimator
T2.2	Large Tau drives posterior mean close to the observation `x_j`

T3: Known Signal Recovery K=1 (3 tests)

Test	What is verified
T3.1	Dense factor: correlation between recovered and true f exceeds 0.7
T3.2	Sparse factor (50/500 nonzero): active features have larger posterior mean than inactive
T3.3	Higher Tau □ better recovery (monotone precision effect)

T4: Multi-Factor K=5 (3 tests)

Test	What is verified
T4.1	Output matrices EF, EF2 have shape p × K
T4.2	All second moments >= squared means (`EF2 >= EF^2`) component-wise
T4.3	All outputs are finite (no NaN or Inf)

T5: Null Factor Shrinkage (2 tests)

Test	What is verified
T5.1	Pure-noise R_k produces factor near zero (point-normal shrinks to zero)
T5.2	Zero EL_k □ A hits floor, B=0 □ EF near zero

T6: Tau Differential Shrinkage (3 tests)

Test	What is verified
T6.1	High-tau features are shrunk less than low-tau features (same x_j)
T6.2	Larger EL2 □ larger A □ smaller s □ less shrinkage for high-signal factors
T6.3	Uniform Tau □ all `s_j` equal (homoscedastic limit, one shared noise level)

T7: Numerical Stability (4 tests)

Test	What is verified
T7.1	All-zero Tau □ A hits floor, no NaN or Inf in output
T7.2	Extreme Tau=1e10 □ no overflow in A, B, x, s
T7.3	Edge case p=1, n=1 □ correct scalar outputs
T7.4	Custom A_floor value is respected (A >= A_floor)

T8: R_k and Gauss-Seidel (2 tests)

Test	What is verified
T8.1	R_k correctly uses EL_k (not another factor's column)
T8.2	`update_F_all` produces finite results with valid second moments across all K

T9: V2.R Consistency (1 test)

Test	What is verified
T9.1	`update_F_k` output matches the corresponding lines in `Supervised_Bayesian_MF_V2.R` (lines 334–340) exactly

---

Demos

File: `demos/demo_update_F.R` Total: **5 scenarios** — narrative output, no PASS/FAIL.

Demo 1: Tau Cancellation Property

Purpose: Demonstrate the core mathematical property that `tau_j` cancels in `x_j` but not in `s_j`.

Setup: Same Y, L, R_k. Run `update_F_k` twice — once with `Tau = rep(1, p)`, once with `Tau = rep(1000, p)`.

Expected output: - `max|x_j(Tau=1) - x_j(Tau=1000)| ≈ 0` (machine precision) - `s_j(Tau=1) / s_j(Tau=1000) ≈ sqrt(1000) ≈ 31.6` for all j

This illustrates that the EBNM "data" is precision-free, while the EBNM "noise level" encodes the precision.

Demo 2: Signal Recovery K=1

Purpose: Confirm that the update recovers a sparse true factor from noisy observations.

Setup: n=150, p=200, K=1. True f_true is sparse (30/200 nonzero entries, drawn from N(0,1)). Y = L %*% t(f_true) + noise.

Reported metrics: - Pearson correlation between `EF[,1]` and `f_true` - Mean posterior mean for active vs inactive features (active should be larger)

Demo 3: Tau-Dependent Shrinkage

Purpose: Show that heteroscedastic precision differentially regularizes features.

Setup: p=200 features split into two groups: first 100 with `tau=10` (clean), last 100 with `tau=0.5` (noisy). Same true signal in both groups.

Reported metrics: - Mean absolute error (MAE) for clean vs noisy features - Posterior SDs for clean vs noisy features - Demonstrates that clean features (high Tau) stay closer to truth

---

Demo 4: Sparsity Recovery

Purpose: Evaluate the point-normal prior's ability to identify which features belong to a sparse factor.

Setup: n=100, p=250, true f is sparse (40/250 nonzero). Threshold posterior means at 0.1 to classify active/inactive.

Reported metrics (confusion matrix):

	Predicted Active	Predicted Inactive
True Active	TP	FN
True Inactive	FP	TN

Also reports precision = TP/(TP+FP) and recall = TP/(TP+FN).

Demo 5: Multi-Factor K=5

Purpose: Validate `update_F_all` on a structured 5-factor problem.

Setup: n=200, p=300, K=5. Each factor has a distinct sparse structure; F is initialized at zero.

Reported metrics: - Column-wise correlations: `cor(EF[,k], F_true[,k])` for each k - Number of active features per factor (posterior mean > 0.1) - Confirms Gauss-Seidel within-loop updates do not break structure

---

Computational Notes

Scalar Broadcast

`sum(EL2_k)` is computed once (a single scalar) and broadcast across all p features via element-wise multiplication with `Tau`. This avoids a p × n matrix multiply for A_F.

Shared Dependency: compute_R_k

`compute_R_k` is defined in `code/update_L.R`. Always source `update_L.R` before `update_F.R`. The residual matrix R_k has shape n × p and represents the data with all other factors' contributions removed. Because L is fully updated before the F step, the R_k computation here uses the most recent L posterior.

Memory Layout

Matrix	Shape	Update order
EL	n × K	Fixed (updated in L step)
EL2	n × K	Fixed (updated in L step)
EF	p × K	Updated column by column (Gauss-Seidel)

Matrix	Shape	Update order
EF2	p × K	Updated column by column
R_k	n × p	Recomputed fresh for each k

For large p, R_k (n × p) dominates memory. It is not cached between factor updates.

Related Files

File	Relationship
`code/update_L.R`	Defines `compute_R_k()` required by this module
`code/update_tau.R`	Provides Tau used as input here
`code/update_beta.R`	Beta update (survival); F does not appear there
`code/Supervised_Bayesian_MF_v2.R`	Main algorithm; lines 334–340 are the reference for T9.1
`tests/test_update_F.R`	26 tests, all passing
`demos/demo_update_F.R`	5 narrative demos
`derivations/qF/qF_update_derivation.tex/.pdf`	Full derivation including tau cancellation proof
`code/SupervisedMF_Context.md`	AI/code quick-reference for the full R implementation